

Experiment No. 7

Aim: Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE.

Theory

1. Overview of Ethereum Development & Local Blockchains

Developing Decentralized Applications (DApps) directly on the Ethereum Mainnet or public testnets presents challenges such as transaction latency, rate limits, and gas expenses. To streamline development, testing, and debugging, developers use **local private blockchains**. A local blockchain simulates the Ethereum Virtual Machine (EVM) on a personal workstation. It provides instantaneous block mining, customizable block times, pre-funded virtual accounts, and deterministic state controls. This sandboxed environment allows developers to iterate rapidly without risking real financial assets.

2. The Truffle Development Suite

Truffle is an industry-standard development environment, testing framework, and asset pipeline for EVM-based blockchains. It manages the complete lifecycle of smart contract development through automated workflows.

Core Capabilities:

- **Built-in Smart Contract Compilation:** Converts human-readable Solidity source code (.sol) into EVM bytecode and Application Binary Interfaces (ABI).
- **Scriptable Migrations Framework:** Facilitates deployment pipelines that track prior deployments directly on-chain, preventing unnecessary redeployments.
- **Network Management:** Manages configurations across distinct environments (local, private consortia, public testnets, and mainnet) via `truffle-config.js`.
- **Interactive Console & Script Execution:** Provides a Node.js-based runtime environment (`truffle console` and `truffle exec`) bound with Web3 abstraction layers for contract interaction.

Truffle Project Architecture:

- `contracts/`: Directory storing all Solidity source files, including the core logic contracts and `Migrations.sol`.
- `migrations/`: Numbered JavaScript migration files (e.g., `1_initial_migration.js`, `2_deploy_contracts.js`) defining deployment sequences and dependency injection.
- `test/`: Directory housing automated JavaScript or Solidity test suites.
- `build/contracts/`: Output folder populated post-compilation containing JSON schema artifacts (the bytecode, compiler metadata, AST, and ABI).
- `truffle-config.js`: Central configuration file detailing RPC host addresses, port mappings, gas parameters, network IDs, and compiler versioning.

3. Ganache: Local EVM Simulation Engine

Ganache serves as the localized ledger within the Truffle Suite. When launched, Ganache instantiates a private, standalone Ethereum blockchain node accessible through the JSON-RPC interface (typically running at `[http://127.0.0.1:7545](http://127.0.0.1:7545)` or `[http://127.0.0.1:8545](http://127.0.0.1:8545)`).

Key Architectural Features:

- **Deterministic Account Generation:** Generates a set of 10 accounts from a standardized

12-word mnemonic phrase, each initialized with 100 dummy Ether (*ETH*).

- **Instant Mining (Automine):** Blocks are mined immediately upon receiving a valid transaction payload, removing the block confirmation delay inherent to public networks.
- **Cryptographic Key Access:** Exposes private keys for all generated accounts, enabling direct import into wallet providers (e.g., MetaMask) for frontend testing.
- **Zero-Gas and Fee Control:** By default, local Ganache instances can run with a gas price of zero or a negligible fraction of a Gwei. This allows smart contract deployments and functional state changes to proceed without significantly depleting account balances, ensuring uninterrupted testing cycles.

4. Smart Contract Compilation, Artifacts, and ABIs

The deployment lifecycle requires converting high-level Solidity code into machine-executable formats:

1. **EVM Bytecode:** The low-level, hexadecimal-encoded opcodes executed directly by the Ethereum Virtual Machine during contract creation and transaction processing.
2. **Application Binary Interface (ABI):** A JSON-formatted interface specification that defines all public functions, arguments, return values, and events within the smart contract. The ABI acts as the communication protocol bridge, allowing external clients (scripts, Web3.js, Ethers.js) to encode function parameters into calldata and decode return data received from the blockchain.

5. Truffle Migrations and Deployment Architecture

Deploying contracts in Truffle differs from simple raw deployments due to the **Migration Mechanism**:

- **Migrations.sol:** An initial tracking contract deployed to the blockchain that maintains a single state variable: `last_completed_migration`.
- **State-Aware Deployment:** When `truffle migrate` executes, Truffle queries the deployed `Migrations` contract. It only executes migration scripts numbered higher than `last_completed_migration`. This ensures stateful persistence, prevents duplicate contract deployments, and saves computational overhead across iterative deployment runs.
- **Deployer Abstraction:** The `deployer` object exposed inside migration scripts handles deployment order, library linking, and constructor parameter passing.

6. Transaction Processing: Calls vs. Transactions

Interactions with deployed contracts on the local node fall into two distinct functional categories:

- **Calls (view / pure functions):**
 - Read-only queries into blockchain storage (e.g., `get()` functions).
 - Executed entirely locally within the EVM node without requiring consensus or block creation.
 - Do not alter global state, consume zero gas, and execute synchronously.
- **Transactions (State-Changing functions):**
 - Mutate the contract's storage variables (e.g., `set(uint x)`).
 - Must be signed by a sender address using its private key.
 - Broadcast to the local network, packaged into a newly mined block, and assigned a unique Transaction Hash (`txHash`).
 - Consume computational units measured as **Gas**, bounded by the block's gas limit.

Install Ganache?

Publisher: Consensys Software Inc.

Version: 2.7.1.0

Source: archive.trufflesuite.com

Capabilities:

- Uses all system resources

Getting the system ready for install...

☐ Launch when ready

Cancel

```
PS C:\Users\heram\Desktop\Exp_7_blockchain> npx truffle compile
```

● >>

Compiling your contracts...

=====

```
> Compiling .\contracts\Migrations.sol
> Compiling .\contracts\SimpleStorage.sol
> Artifacts written to C:\Users\heram\Desktop\Exp_7_blockchain\build\contracts
> Compiled successfully using:
  - solc: 0.8.20+commit.a1b79de6.Emscripten.clang
```

Compiling your contracts...

=====

```
> Compiling .\contracts\Migrations.sol
> Compiling .\contracts\SimpleStorage.sol
> Artifacts written to C:\Users\heram\Desktop\Exp_7_blockchain\build\contracts
> Compiled successfully using:
  - solc: 0.8.20+commit.a1b79de6.Emscripten.clang
```

Starting migrations...

=====

```
> Network name: 'development'
> Network id: 5777
> Block gas limit: 6721975 (0x6691b7)
```

1_initial_migration.js

=====

Replacing 'Migrations'

```
> transaction hash: 0x7f8bd17c70dd2bded2987e5042b4dc55af30c5bf71d60320312e8eeb2467a335
> Blocks: 0 Seconds: 0
> contract address: 0x5c6ec88ff9A3A9677F1e01B4f894F87Fb84C4fff6
> block number: 6
> block timestamp: 1791389013
> account: 0xc8637eBF4F8e148B2CAe4Bf5D0C844D60836c2C7
> balance: 99.99764469459288602
> gas used: 250154 (0x3d12a)
> gas price: 2.958288746 gwei
> value sent: 0 ETH
```

```
> gas price:      2.958288746 gwei
> value sent:     0 ETH
> total cost:     0.000740027762966884 ETH
```

```
> Saving migration to chain.
> Saving artifacts
-----
```

```
> Total cost:     0.000740027762966884 ETH
```

2_deploy_contracts.js

=====

Replacing 'SimpleStorage'

```
> Blocks: 0      Seconds: 0
> contract address: 0xe25074697d309198a22dB4E8256893439f56C4fE
> block number: 8
> block timestamp: 1791389013
> account: 0xc8637eBF4F8e148B2CAe4BF5D0C844D60836c2C7
> balance: 99.997152528072755338
> gas used: 125653 (0x1ead5)
> gas price: 2.855300109 gwei
> value sent: 0 ETH
> total cost: 0.000358777024596177 ETH
```

```
> Saving migration to chain.
> Saving artifacts
-----
```

```
> Total cost:     0.000358777024596177 ETH
```

Summary

=====

```
> Total deployments: 2
> Final cost:        0.001098804787563061 ETH
```

PS C:\Users\heram\Desktop\Exp_7_blockchain> █

● PS C:\Users\heram\Desktop\Exp_7_blockchain> npx truffle exec scripts/interact.js

>>

This version of μWS is not compatible with your Node.js build:

Error: Cannot find module '../binaries/uws_win32_x64_137.node'

Require stack:

- C:\Users\heram\Desktop\Exp_7_blockchain\node_modules\ganache\node_modules\@trufflesuite\uws-js-unofficial\src\uws.js
- C:\Users\heram\Desktop\Exp_7_blockchain\node_modules\ganache\dist\node\core.js
- C:\Users\heram\Desktop\Exp_7_blockchain\node_modules\truffle\build\exec.bundled.js
- C:\Users\heram\Desktop\Exp_7_blockchain\node_modules\original-require\index.js
- C:\Users\heram\Desktop\Exp_7_blockchain\node_modules\truffle\build\cli.bundled.js

Falling back to a NodeJS implementation; performance may be degraded.

Using network 'development'.

```
-----
Connected to deployed SimpleStorage at: 0xe25074697d309198a22dB4E8256893439f56C4fE
Executing transaction: set(100)...
Transaction Hash: 0xa9a2e71951a3b49445534ec1f11c6c3b9878de85df8dac95791bbf23d445e6a2
Block Number: 10
Gas Used: 43702
Calling get()...
Current Stored Value: 100
-----
```

● PS C:\Users\heram\Desktop\Exp_7_blockchain> █

ACCOUNTS	BLOCKS	TRANSACTIONS	CONTRACTS	EVENTS	LOGS	SEARCH FOR BLOCK NUMBERS OR TX HASHES
CURRENT BLOCK 10	GAS PRICE 20000000000	GAS LIMIT 6721975	HARDFORK MERGE	NETWORK ID 5777	RPC SERVER HTTP://127.0.0.1:7545	MINING STATUS AUTOMINING
WORKSPACE QUICKSTART						SAVE SWITCH
TX HASH 0xa9a2e71951a3b49445534ec1f11c6c3b9878de85df8dac95791bbf23d445e6a2						
FROM ADDRESS 0xc8637eBF4F8e148B2CAe4BF5D0C844D60836c2C7		TO CONTRACT ADDRESS 0xe25074697d309198a22db4E8256893439f56C4fE		GAS USED 43702	VALUE 0	CONTRACT CALL
TX HASH 0x3906b6d67c952c9507217a5adbb94f01ae018486c07a7285e25c2de3849a2ad0						
FROM ADDRESS 0xc8637eBF4F8e148B2CAe4BF5D0C844D60836c2C7		TO CONTRACT ADDRESS 0x5c6ec88fF9A3A9677F1e01B4f894F87Fb84C4ff6		GAS USED 28813	VALUE 0	CONTRACT CALL
TX HASH 0xfd879629e51bf919d48a5f9cd8922ea942b81bf7c509c099082dcf5261c00d09						
FROM ADDRESS 0xc8637eBF4F8e148B2CAe4BF5D0C844D60836c2C7		CREATED CONTRACT ADDRESS 0xe25074697d309198a22db4E8256893439f56C4fE		GAS USED 125653	VALUE 0	CONTRACT CREATION
TX HASH 0x42b026b8c4d923d437366b9bc77ccfc929f88fdce4863224dd4f1d55eaa1a1c8						
FROM ADDRESS 0xc8637eBF4F8e148B2CAe4BF5D0C844D60836c2C7		TO CONTRACT ADDRESS 0x5c6ec88fF9A3A9677F1e01B4f894F87Fb84C4ff6		GAS USED 45913	VALUE 0	CONTRACT CALL
TX HASH 0x7f8bd17c70dd2bded2987e5042b4dc55af30c5bf71d60320312e8eeb2467a335						
CONTRACT CREATION						

ACCOUNTS	BLOCKS	TRANSACTIONS	CONTRACTS	EVENTS	LOGS	SEARCH FOR BLOCK NUMBERS OR TX HASHES
CURRENT BLOCK 10	GAS PRICE 20000000000	GAS LIMIT 6721975	HARDFORK MERGE	NETWORK ID 5777	RPC SERVER HTTP://127.0.0.1:7545	MINING STATUS AUTOMINING
WORKSPACE QUICKSTART						SAVE SWITCH
BLOCK 10	MINED ON 2026-10-07 21:35:02				GAS USED 43702	1 TRANSACTION
BLOCK 9	MINED ON 2026-10-07 21:33:34				GAS USED 28813	1 TRANSACTION
BLOCK 8	MINED ON 2026-10-07 21:33:33				GAS USED 125653	1 TRANSACTION
BLOCK 7	MINED ON 2026-10-07 21:33:33				GAS USED 45913	1 TRANSACTION
BLOCK 6	MINED ON 2026-10-07 21:33:33				GAS USED 250154	1 TRANSACTION
BLOCK 5	MINED ON 2026-10-07 21:29:45				GAS USED 43702	1 TRANSACTION
BLOCK 4	MINED ON 2026-10-07 21:29:31				GAS USED 28813	1 TRANSACTION
BLOCK 3	MINED ON 2026-10-07 21:29:31				GAS USED 125653	1 TRANSACTION
BLOCK 2	MINED ON 2026-10-07 21:29:31				GAS USED 45913	1 TRANSACTION
BLOCK 1	MINED ON 2026-10-07 21:29:30				GAS USED 250154	1 TRANSACTION

Conclusion

The experiment was successfully executed by establishing a local Ethereum development pipeline using the Truffle Suite and Ganache. The smart contract was written, compiled into EVM bytecode and ABI artifacts, and migrated onto the local private blockchain running at **127.0.0.1:7545**. Through automated script execution, state-changing write transactions and view-based read calls were verified, confirming state persistence and block creation on the local ledger without incurring real financial gas fees. This practical exercise established a complete foundational understanding of local node configuration, automated migration scripts, and decentralized application interaction workflows.